

Psyche Profits: Web-Based Game

Nathan Esber(Team leader)

Dante Hampton

Sai Nithin Guduru

Mohammed LNU

Team 13

Department of Computer Science, Cleveland State University

Submitted to— NASA Psyche Mission

Contents

Executive Summary	3
1. Statement of Problem and Background	4
2. Design Objectives	4
3. Technical Approach	4
3.1 Identifying the Unmet Needs	5
3.2 Determining the Design Constraints	5
3.3 Defining the Technical Specifications	5
3.4 Enumerating the Design Concepts	6
3.5 Standards Compliance	7
4. Detailed Design	7
4.1 Scenes	7
4.1.1 Mining Scene	8
4.1.2 Spaceship Scene	9
4.1.3 Other Scenes	10
4.2 Pseudocode	11
4.2.1 CurrencyManager.cs	11
4.2.2 AsteroidSlotManager.cs	12
4.2.3 Drill.cs	12
4.2.4 ClickableHighlight.cs	13
4.2.5 WinConditionButton.cs	14
5. Verification	15
5.1 Browser Compatibility Testing	16
5.2 Functional Verification	17
5.3 User Testing	17
5.4 Educational Verification	18
5.5 Sponsor Constraint Compliance	18
6. Project Managements	19
6.1 Team Qualifications	19
6.2 Task Assignment	20
6.3 Timeline	20
6.4 Deliverables	21
6.5 Budget	21
6.6 Communication and Coordination with Sponsor	22
7. Professional Awareness	22
8. Conclusion	23
References	24
Appendix A: Résumés of Team Member	25
Nathan Esber	26
Mohammed Arshad	27
Dante Hampton	27
29	
Appendix B: Sponsor specification	30

Executive Summary

NASA's Psyche mission is a groundbreaking expedition to explore the metal rich asteroid Psyche, with the spacecraft expected to arrive in mid-2029. Our group has been tasked to create a 2-dimensional interactive web-based game to connect users to the mission. The game should be able to run without the connection of other players and should be able to work on consumer-grade hardware and the internet. The game should actively avoid presenting misleading or inaccurate information. If the setting takes place in the far future or involves speculative elements, this should be clearly communicated to the player to prevent confusion with real-world facts. Psyche Profits is a single player web-based video game with the aim to raise excitement about the upcoming Psyche mission. The user is tasked with extracting resources called "Materials" from the asteroid, converting them into credits, and upgrading their assets. Once the user has met all three requirements, they can have the option to continue playing or win the game. The game ensures to display facts about the mission in the "Fun Fact" menu. When users hear the phrase "educational game" many of them are turned off. Players would rather play a game with educational elements than an educational game with fun elements. Overall, the design aims to balance an engaging gameplay experience with educational elements.

1. Statement of Problem and Background

The goal of this project is to explain NASA's current ongoing mission to the asteroid Psyche. Psyche is a metal rich asteroid containing nickel and iron located in the asteroid belt between Mars and Jupiter. The problem that this game solves is being able to get the attention of a wide demographic of people and inform them of the mission in an entertaining way. This video game is an economical way for NASA to spread awareness of the mission without expending too much financial capital. Psyche Profits can be scaled to run on multiple servers to handle player demands as needed.

2. Design Objectives

The design objectives fluctuated multiple times through the development of the project. The primary objectives that have stayed the same are ensuring accessibility through simple browser-based deployment. The game should allow users to use a variety of different browsers such as Google Chrome, Microsoft Edge, Firefox, and Mozilla to access our application. The game should not introduce or reinforce misconceptions; players should be aware that it is taking place in the far future. Players should be able to play the game without the connection of other users, allowing for a single player experience. The game is aimed at engaging the user with excitement and innovation of the Psyche mission. Finally, developing a stable core gameplay system that supports progression and mission success. The technical foundation of the application includes resource tracking, upgrades, economic balancing, and performance optimization.

3. Technical Approach

This section outlines the overall technical approach used in the design of Psyche Profits. It identifies unmet needs, establishes design constraints, and defines the technical specifications that guide development. Additionally, it explores the potential design concepts, evaluates and selects the most appropriate solutions, and ensures compliance with relevant standards.

3.1 Identifying the Unmet Needs

Factual or information heavy content can reduce player engagements; many players prioritize entertainment over dense educational material. This creates an unmet need for more engaging and relatable experiences. Another unmet need is the lack of engagement among those who are typically not interested in video games. This highlights the importance of designing an experience that is accessible, intuitive, and appealing to the broader audience. The Cleveland Clinic has indicated that for some users, video games can be as addictive as gambling, in which the rush of winning becomes one of the main reasons for playing. Recent neurological research shows similarities in the brains of people with videogame addiction and substance use disorder.

3.2 Determining the Design Constraints

A list of constraints was given to us by our sponsor. The documentation made it extremely clear that the game should not include shooting or violence of any sort. It should not introduce or reinforce misconceptions; if our game has something that is not realistic into the current Psyche mission, the game should make it clear that it is taking place in the far future. The system should not support in-game purchases using real-world currency, avoiding the inclusion of e-commerce features. The system should not allow the user to make accounts or profiles. The players should not have any way to contact each other and do not infringe copyrights or misuse licenses. The system should not harm the user's system or have any security vulnerabilities.

3.3 Defining the Technical Specifications

We used Unity as our primary development environment. The group struggled to learn Unity as first, as it was extremely overwhelming to have so many tools knowing very little about game development. Unity allowed for a stable platform for building interactive systems, managing game objectives, handling graphic user interfaces, integrating artwork, and logic.

We considered using Godot because our group was more familiar with Python, but after some consideration our group decided to commit to unity. This was because of several different reasons, many of the game engines can do the exact same thing so it came down to preference. Unity has been around for a much longer time and has a large community. Many of the problems or issues we had were easily solved by doing a quick internet search.

Gameplay mechanics are composed of a variety of different systems. Psyche Profits is a resource and economic management simulation game, where the user is tasked with making profits from Psyche. Psyche Profits is created using a 2-dimensional pixel art style. All the assets were found online for free or using the unity asset store. The users use drills to generate material, which can then be used to convert into credits or used to expand and upgrade the ship. Asteroid showers are an event to slow the progression down of the user. In the Heads-Up-Display (HUD) the user can select the “Fun Fact” button which will show a variety of fun facts taken directly from the official Psyche website.

3.4 Enumerating the Design Concepts

Here is a list of design concepts that were scrapped when making the game. These were listed in the game proposal but did not make the final cut.

Design Concept	Short Summary of the Concept
Hire Crew to Manage Drills	The team wanted to add astronauts into the game with the intention that the user could hire them to access the drills. We decided to scrap this idea mainly due to the lack of free assets. Many of the assets also did not match our pixel art system or include weapons, which are against our constraints.
Day and Night Cycle	Psyche experiences a day and night cycle. We intended to add into the game for an

	educational element but did not have the time for it. We concluded on adding to the Fun Facts tab
Energy Management	The team originally wanted to build upon the day and night cycle by adding an energy system you would have to manage when it was nighttime. Due to the scrapping of the day and night cycle we decided to scrap the energy management system. The team intended the game to be played by all ages, keeping the game relatively easy and simple is a crucial factor.

Figure 1: Design Concepts that were scrapped.

3.5 Standards Compliance

The industrial standards clearly state the protection of consumers, as well as their data privacy. This can be seen as the CFAA, Computer Fraud, and Abuse Act, from a government perspective. The players should not have a way to contact each other, preventing malicious intended activities from spreading or being seen. This prevents the use of methods such as phishing, drive-by-downloads, and harassment. The game should not have security vulnerabilities. The game should not introduce or reinforce misconceptions. Psyche Profits is not realistic to the current Psyche mission; a disclaimer is made in the tutorial claiming the mission takes place in the year 2100+.

4. Detailed Design

4.1 Scenes

Psyche Profits is organized into multiple interconnected scenes, each responsible for a specific aspect of the overall functionality. The scenes work together to manage gameplay flow, user interaction, and system behavior. An overview of the structure is illustrated in Figure 2, which highlights how control transitions between scenes and how each contributes to the overall system of operation.

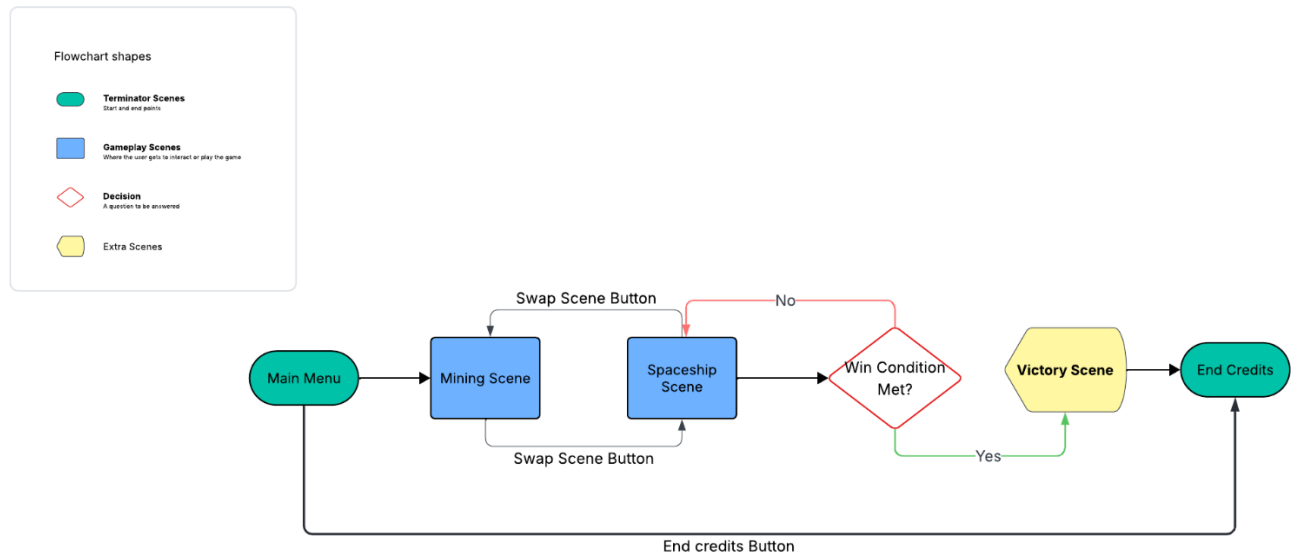


Figure 2: 5 different scenes the makeup Psyche Profits

4.1.2 Mining Scene

The mining scene is the main scene where the user interacts with the Psyche Profits. Here the user can be first prompted by a tutorial, which gives them a little background about the mission and what each button does on the HUD. The user must have the required number of credits to place the drill, otherwise the drill will not be placed. Once placed, the drill will call the drill script which is attached to each drill in the game. It manages the drills' speed and depth levels, generates material over time, and manages the upgrade system for both attributes. Once the drill is placed, the system will open another drill slot with a max number being five. The user can upgrade their drill a total of two times in both speed and depth, once doing so they will unlock a golden drill. Upgrading all the drills and using all the drill slots is one of the requirements needed for the win condition.

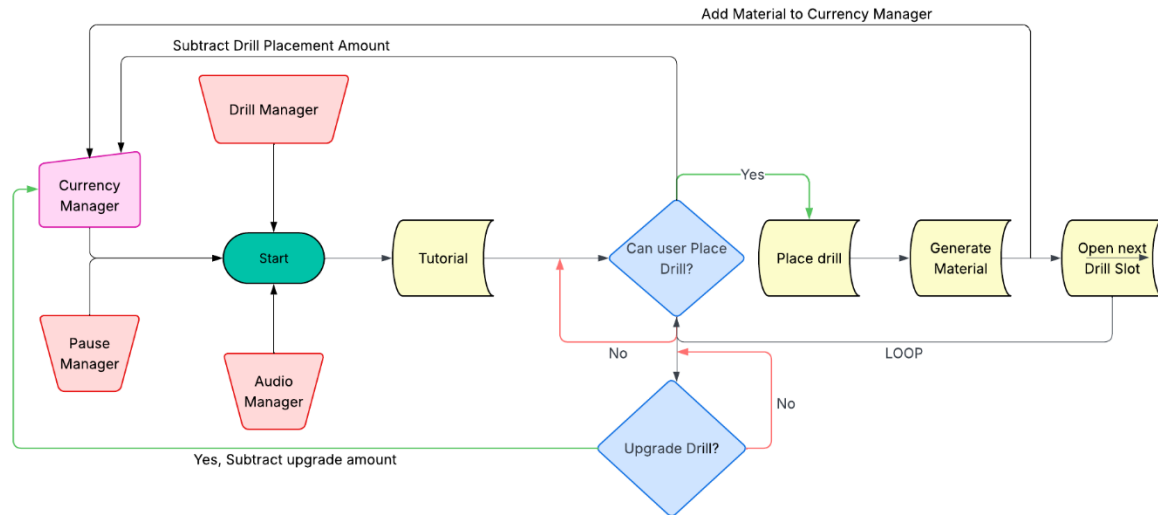


Figure 3: Mining Scene Overview

4.1.3 Spaceship Scene

The spaceship scene is the secondary scene for Psyche Profits. In this scene, the user is allowed to upgrade their ship as this is one of the three requirements for the win condition. The user can also be encountered by an asteroid shower! This system is meant to slow down the progression of the player; the user will have to decide to fix the repairs immediately or add the repair damage to the next upgrade cost. Players will have to use material and credits, unlike the drill, to upgrade the ship.

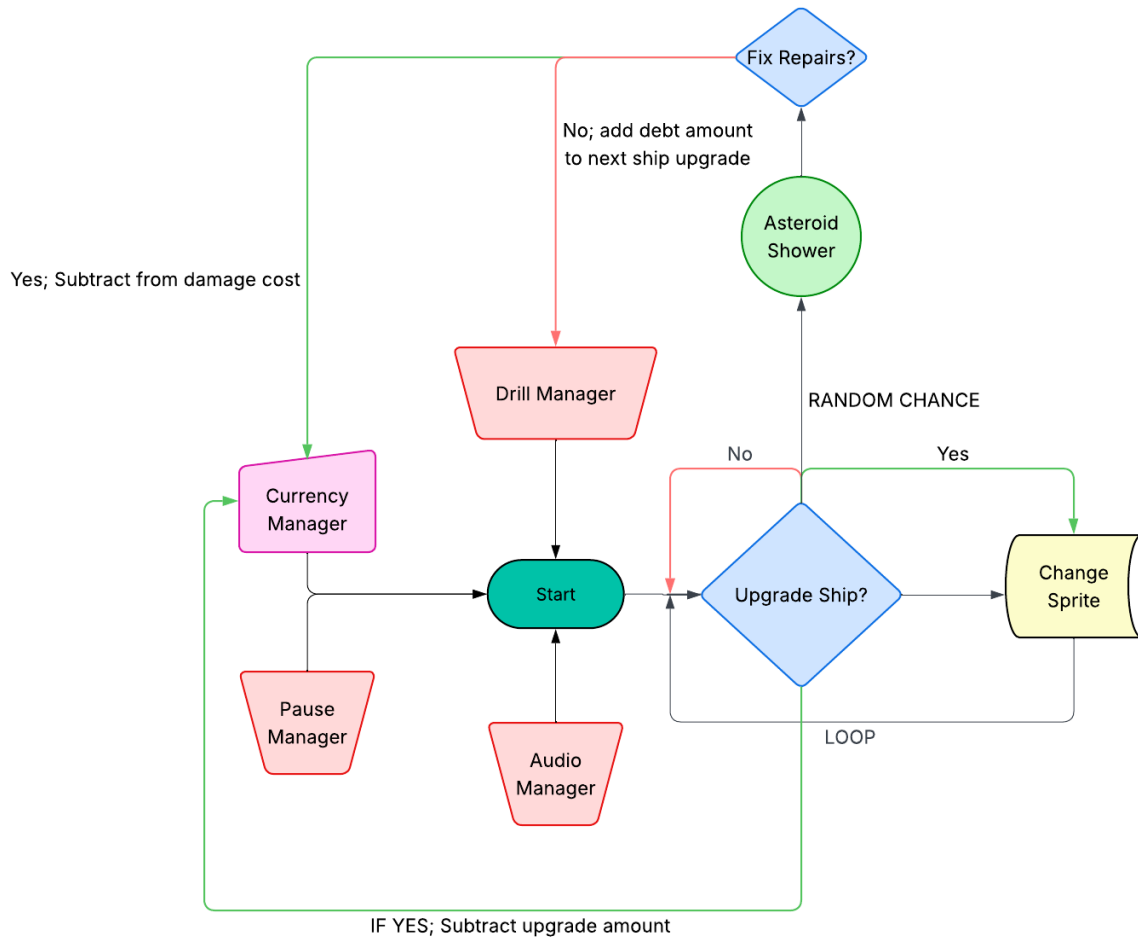


Figure 4 Spaceship Scene Overview

4.1.4 Other Scenes

The rest of the scenes, such as the main menu, victory scenes, and end credits, are self-explanatory. When the user loads the game, they will be brought to the main menu. Here the user can start the game, being brought to the main menu. The user can select end credits to bring them to the credit scene, or they can select the exit button. The victory scene is only shown to the player when the player has met all 3 conditions and selects win. Finally, the victory scene will send the player to the end credits scene which shows the references and credits.



Figure 5 Main Menu of Psyche Profits

4.2 Pseudocode

Psyche Profits is built upon hundreds of lines of code that form the core foundation of the system. Several key scripts play important roles in the overall functionality including, CurrencyManager.cs , AsteroidSlotManager.cs, Drill.cs, ClickableHighlight.cs, and WinCondition.cs. Each of these scripts is responsible for handling specific aspects of system behavior. The following section outlines their functionality through pseudocode description.

4.2.1 CurrencyManager.cs

CurrencyManager is responsible for keeping track of the materials and credits the user has.

This script initializes a single shared instance, so all the scripts can refer to one currency manager. This allows the user to buy upgrades and convert material into credits.

Initialize a single shared CurrencyManager
Set starting currency to 500

Function AddCurrency(amount)
 Increase currency

Function SpendCurrency(amount)
 If not enough currency
 Return failure
 Otherwise
 Deduct currency
 Return success

Figure 6 Pseudocode for Currency Manager

4.2.2 AsteroidSlotManager.cs

When the user places a drill, another drill slot appears for the users to select. Clicking on the drill slot will open the drill placement menu prompting the user with the question “Do you want to place the drill for X number of credits”. If the user selects yes, then the system calls Drill.cs to place the drill. After the drill is placed a new drill slot opens.

On game start
 Restore previously placed drills (for when reloading the scene)
 Activate the next available slot

Function RestoreDrills
 For each slot
 Check if saved drill data exists
 If it exists
 Mark slot as occupied
 Recreate the drill with saved upgrades
 Otherwise
 Mark slot as empty

Function ActivateNextSlot
 Find the first unoccupied slot
 Activate that slot

Function OnSlotUsed(slot)
 Find the index of the used slot
 Activate the next slot in the list

Figure 7 Pseudocode AsteroidSlotManager.cs

4.2.3 Drill.cs

The Drill is responsible for generating materials over time and upgrading its performance through speed and depth levels. It also updates its appearance when it is fully upgraded. Speed level controls how fast materials generate and depth level controls how many materials are generated.

```

On start
Set speed and depth stats based on current levels
Determine placement cost

Every frame
Increase timer
If timer reaches interval
Generate materials
Reset timer

Function GenerateMaterial
Generate random amount based on depth level
Add materials to player

Function UpgradeSpeed
If max level reached OR not enough currency
Exit
Increase speed level
Update speed stats
Save drill data

Function UpgradeDepth
If max level reached OR not enough currency
Exit
Increase depth level
Update depth stats
Save drill data

If both upgrades reach max level
Change drill appearance (golden)

```

Figure 8 Pseudocode for Drill.cs

4.2.4 ClickableHighlight.cs

This script handles player interaction with a clickable object that can be upgraded through multiple levels. It manages highlighting, upgrading UI, resource checking, visual changes, and camera zoom progression.

```

On start
  Load current upgrade level
  Apply correct visual sprite
  Restore saved camera zoom

On mouse hover
  Highlight object
On mouse exit
  Remove highlight

On left click
  If object is clicked
    Show upgrade popup

Function ShowUpgradePopup
  If max level reached
    Exit

  Calculate next upgrade costs
  Include debt penalty if applicable

  Display cost information
  Enable or disable upgrade button based on resources

Function OnYesClicked (Upgrade confirmed)
  Check if player has enough resources
    If not enough
      Close popup and exit

  Deduct materials and currency
  Remove any debt penalty

  Increase upgrade level
  Save level progress

  Update visual sprite

```

Figure 9 Pseudocode ClickableHighlight.cs

4.2.5 WinConditionButton.cs

This script checks whether the player has met all requirements to win the game. It evaluates drill progress, ship level, and total currency, then updates the UI and enables the win option when all conditions are satisfied.

```

Function CheckWinCondition
    If drill system exists
        If all slots are filled AND all drills are fully upgraded
            Set drill condition = true
        Else
            Set drill condition = false

    If ship level is 5 or higher
        Set ship condition = true
    Else
        Set ship condition = false

    If currency >= required amount
        Set credit condition = true
    Else
        Set credit condition = false

    Update UI indicators for each condition

    Show win condition interface

    If all conditions are true
        Enable win button
    Else
        Disable win button

Function ConfirmWin
    If all conditions are true
        Load victory scene

```

Figure 10 Pseudocode for CheckWinCondition

5.Verification

The verification of Psyche Profits was evaluated against two primary goals: confirming that the game operates correctly and without issues and confirming that it achieves its educational purpose of spreading awareness about NASA's Psyche mission.



Figure 11 Main Menu

5.1 Browser Compatibility Testing

The game was evaluated across multiple standard web browsers as required by the sponsor specification. The game ran successfully on Google Chrome, Microsoft Edge, Firefox, and Mozilla-based browsers without any reported issues. Users were able to load, navigate, and interact with the game consistently across all tested platforms on standard consumer-grade hardware and home Wi-Fi connections, meeting the sponsor's accessibility requirements.



Figure 12 Win Condition GUI

5.2 Functional Verification

The game was verified to be fully functional and playable in a web browser without any crashes or errors. All core gameplay systems including the drill placement and upgrade mechanics, the currency and materials management system, the spaceship upgrade scene, and the asteroid shower event operated as intended. The win condition logic was also confirmed to correctly evaluate all three requirements (all drill slots filled and upgraded, ship upgraded to level 5 or higher, and sufficient credits accumulated) before enabling the victory screen.

5.3 User Testing

The game was shared with a group of users including classmates and friends via a browser link. Testers were able to play through the game without encountering any bugs, crashes, or confusing interactions. No errors were reported during playtesting. User feedback indicated a positive response to the gameplay experience, and multiple testers expressed interest and excitement in learning more about NASA's Psyche mission after playing.



Figure 13 Fun facts GUI

5.4 Educational Verification

The educational objectives of the game were met through the inclusion of a "Fun Facts" menu accessible from the HUD during gameplay. This feature presents information sourced directly from the official NASA Psyche mission website, allowing players to learn about the mission in a non-intrusive way. Additionally, the game includes a tutorial disclaimer clearly stating that the game's events take place in the year 2100+, ensuring players are not misled about the current state of the mission. These elements together satisfy the sponsor requirement that the game not introduce or reinforce misconceptions.

5.5 Sponsor Constraint Compliance

A review of the final game confirmed compliance with all constraints outlined in the sponsor's specification. The game contains no shooting or violence, no real-money transactions, no user accounts or player-to-player communication, and no copyrighted or improperly licensed content. The game does not pose any security risks or cause harm to the user's system.

Overall, Psyche Profits successfully meets both its functional and educational verification

criteria, delivering a stable, accessible, and informative experience that aligns with the goals of the NASA Psyche mission outreach effort.

6. Project Managements

6.1 Team Qualifications

Detailed resumes for all team members are included in Appendix A.

Our team consists of students with backgrounds in computer science, software development, and web technologies. Each member brings different strengths that directly support building a web-based tycoon/economy game. Our skills include programming in JavaScript, Python, and Java, experience with front-end and back-end development, and familiarity with game logic, UI design, and basic project management.

Nathan Esber is the group leader of the project, having a credible background when it comes to software development. He has worked on different personal projects such as Minecraft servers using Java, and experiences with other languages such as Python and C through university.

Dante Hampton has experience in programming languages such as Java, Python, C, and other OOP languages. He has worked on a wide variety of projects ranging from videogame design to VM hypervisor and website development. He also has experience programming various microcontrollers and IoT devices.

Sai Nithin Guduru contributes experience in software development, databases, and front-end design. He has worked on several academic projects using Python, Java, and JavaScript, and has practical skills in building interactive applications. For this project, he supports game logic, UI structure, and overall organization of the development workflow, helping the team keep features functional and on schedule.

Mohammed LNU contributing to this project has a strong background in software development, particularly in Java, Python, and modern web technologies. He has experience

building interactive applications and handling data-driven systems. As an engineering peer teacher and a research assistant, he has gained practical experience explaining technical concepts and working on structured projects.

6.2 Task Assignment

The group used an Excel spreadsheet to track all required tasks. The spreadsheet was colored to prevent multiple members from working on the same task simultaneously. A task highlighted in green means the task has already been completed, while yellow signified that the current task is in progress and still requires work. Finally, tasks highlighted in red for the task were either scrapped or needed immediate attention.

Each task entry also included a designated section identifying the team member responsible, along with an additional section for notes to provide updates or important details.

6.3 Timeline

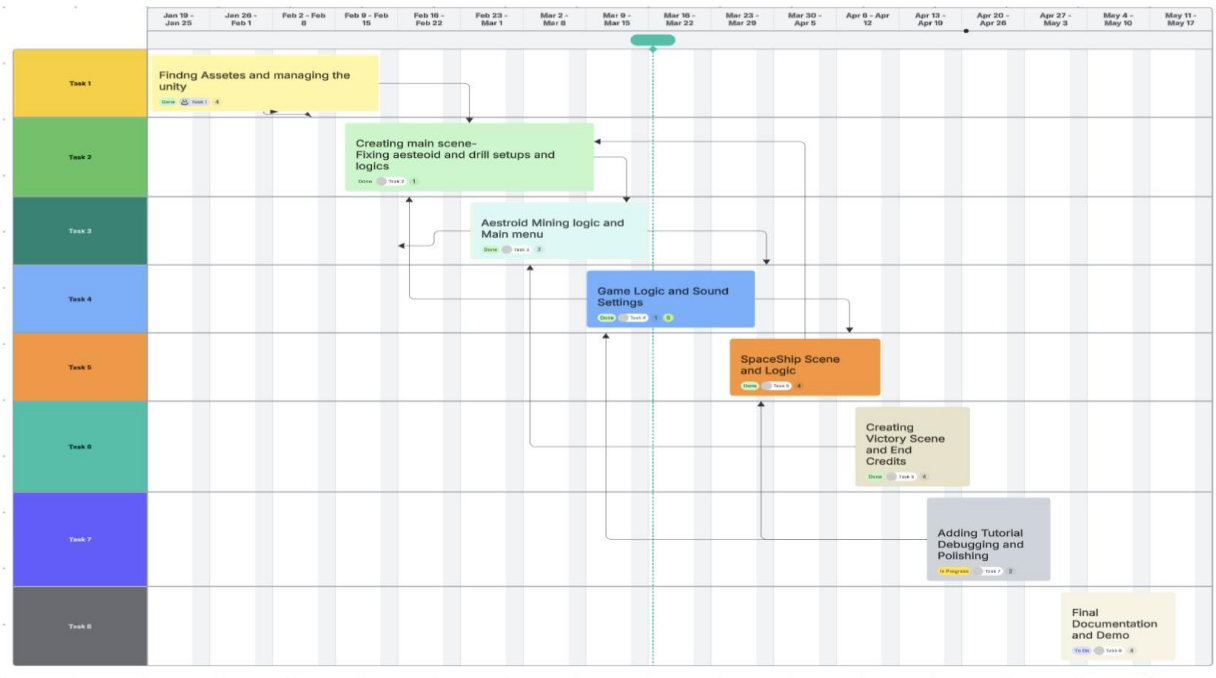


Figure 14 Timeline

6.4 Deliverables

The deliverables for the space mining game project include a functional 2D game developed using Unity and C#, incorporating core gameplay mechanics such as resource extraction, currency management, upgrades, and event-based challenges. The project also includes a complete user interface system with components such as the heads-up display (HUD), pause menu, tutorial system, and interactive controls that support gameplay.

In addition, multiple game scenes were developed, including the primary asteroid mining environment and supporting scenes such as the spaceship interface. Several gameplay systems were implemented to enhance the user experience, including disaster events, progression mechanics, and upgrade features.

The deliverables also include the full source code for the project, maintained using version control tools to support collaboration and track development progress. Supporting documentation was prepared to describe the planning, design, and implementation of the system, along with a final report summarizing the overall project outcomes. Presentation materials were also created to communicate the project concept, technical approach, and key results.

6.5 Budget

There is no major budget requirement because of the nature of writing code. We are not responsible for hosting the website; it is hosted by NASA Psyche Mission, along with the previous projects. Our budget was predicted to be \$0-\$50, and we ended up spending a total of \$0.

Service	Cost
Unity License	\$0

Hosting	\$0
Assets	\$0
Total	\$0

Figure 15 Budget Table and Total Spending

6.6 Communication and Coordination with Sponsor

Our team was onboarded to the communication platform Stack, which served as the primary channel for direct interaction with our project sponsor. This platform provides real-time messaging capabilities allowing us to ask questions and receive quick responses as needed. In addition to this, the sponsor has provided a dedicated Zoom meeting link for virtual discussions and their email contact information for formal communication. These meetings were held every other week from February 3rd to April 28th.

7. Professional Awareness

The development of the space mining game also provided important insights into professional awareness within the field of software engineering.

Throughout the project, it became clear that technical skills alone are not sufficient; factors such as teamwork, communication, ethics, and responsibility play a significant role in the success of any project.

One key aspect of professional awareness demonstrated in this project was teamwork and collaboration. Working in a group requires clear communication, task distribution, and accountability. Each team member was responsible for specific components, and regular meetings ensured that progress was tracked, and issues were addressed early.

Another important consideration was time management and project planning. The team initially proposed a highly ambitious set of features, but as development progressed, it became necessary to reassess priorities and adjust the scope.

Finally, this project increased awareness of professional expectations in software development, including the use of tools such as version control systems, collaborative platforms, and structured workflows. These practices mirror industry standards and prepare

us for future roles in the field.

Overall, the project reinforced that being a professional in software engineering involves more than just coding—it requires effective communication, ethical responsibility, adaptability, and a commitment to producing high-quality work.

8. Conclusion

The development of the NASA Psyche-themed space mining game provided valuable experience in applying software engineering and game development principles in a practical setting. Over the course of the project, our team worked to design and implement core gameplay systems like resource extraction, economic management, upgrades, and event-based challenges. These elements are combined to create an interactive system.

Teamwork has another critical factor in the project's success. By dividing the tasks, maintaining communication, and regularly reviewing progress, we were able to stay aligned and complete the core objectives. At the same time, this experience emphasized the challenges of group projects, including time management and adapting to unexpected setbacks.

Although the game is functional, there is still room for improvement. Future enhancements could include more advanced mechanics, better balancing, improved visuals, and expanded gameplay features. Despite these limitations, the project successfully demonstrates our ability to take a concept from planning to implementation.

Overall, this project was a valuable learning experience that strengthened both our technical and collaborative skills, while also providing insight into the challenges of real-world software development.

References

- [1] “9-48.000 - computer fraud and abuse act,” Justice Manual | 9-48.000 - Computer Fraud and Abuse Act | United States Department of Justice, <https://www.justice.gov/jm/jm-948000-computer-fraud> (accessed Nov. 24, 2025).
- [2] “2d Pixel Asteroids: 2d Environments: Unity Asset Store.” *2D Environments* | *Unity Asset Store*, EVPO Games, assetstore.unity.com/packages/2d/environments/2d-pixel-asteroids-136477. Accessed 20 Apr. 2026.
- [3] “2D Spaceships Free Trial: 2d Environments: Unity Asset Store.” *2D Environments* | *Unity Asset Store*, Daniel Kole Productions, assetstore.unity.com/packages/2d/environments/2d-spaceships-free-trial-146307. Accessed 20 Apr. 2026.
- [4] “2D Spaceships Free Trial: 2d Environments: Unity Asset Store.” *2D Environments* | *Unity Asset Store*, MSGDI, assetstore.unity.com/packages/2d/environments/2d-spaceships-free-trial-146307. Accessed 20 Apr. 2026.
- [6] Code of ethics - international game developers association (IGDA), <https://members.igda.org/page/codeofethics> (accessed Nov. 24, 2025).
- [7] “Godot vs unity: Essential features for game developers - blog - meshy,” Meshy AI - The #1 AI 3D Model Generator, <https://www.meshy.ai/blog/godot-vs-unity> (accessed Dec. 5, 2025).
- [8] Marketplace, Ignite. “Download an 8 Bit Retro Styled Pixel Art Illustration of a Golden Drill. for Free.” *Vecteezy*, www.vecteezy.com/png/19797305-an-8-bit-retro-styled-pixel-art-illustration-of-a-golden-drill. Accessed 20 Apr. 2026.
- [8] Marketplace, Ignite. “Download an 8-Bit Retro-Styled Pixel-Art Illustration of an Iron Drill. for Free.” *Vecteezy*, www.vecteezy.com/png/27226493-an-8-bit-retro-styled-pixel-art-illustration-of-an-iron-drill.

illustration-of-an-iron-drill. Accessed 20 Apr. 2026.

[9] M. Jayasri, “Unity vs. Godot: A game developer’s guide,” DEV Community, <https://dev.to/manasajayasri/unity-vs-godot-a-game-developers-guide-2a6o> (accessed Dec. 5, 2025).

[10] “NSPE code of Ethics for Engineers: National Society of Professional Engineers,” NSPE Code of Ethics for Engineers | National Society of Professional Engineers, <https://www.nspe.org/career-growth/nspe-code-ethics-engineers> (accessed Nov. 24, 2025).

[11] “Pixel Skies Demo Background Pack: 2d Environments: Unity Asset Store.” *2D Environments* | *Unity Asset Store*, Digital Moons, assetstore.unity.com/packages/2d/environments/pixel-skies-demo-background-pack-226622. Accessed 20 Apr. 2026.

[12] “Psyche,” NASA, <https://science.nasa.gov/mission/psyche/> (accessed Nov. 24, 2025).

[13] “Psyche Web-Based Games Guidelines - Platinum Class”, (accessed Nov. 24, 2025).

[14] Shop Empire 2. LittleGiantWorld. Flash/browser game. Released July 6, 2012

[15] “Welcome to the psyche mission’s web-based game portal!,” NASA Psyche Capstone Web Projects Portal, <https://psyche-capstone-web-projects.web.app/> (accessed Nov. 24, 2025).

Appendix A: Résumés of Team Member

The following pages present one-page résumés of the team members for this project.

Nathan Esber

Nathan Esber

n.esber@vikes.csuohio.edu
nathanesber18@gmail.com

Permanent Address
10380 Barr Road
Brecksville, Ohio 44141

Campus Address
1750 Euclid Ave 1115B
Cleveland, OH 44115
Cell: (440) 241-2589

EDUCATION: **Cleveland State University, Cleveland, Ohio** **8/2022-Present**
Candidate for Bachelor of Science in Computer Science
Expected Graduation: 2026

Brecksville - Broadview Heights High School, Brecksville, Ohio **06/2022**
AP Computer Science Principles

SKILLS: **Languages:** Python, C, Java
Operating Systems: Linux, Windows
Coursework: Database Concepts, Computer Security, Data Science, Data Structures, Object Oriented Programming, Operating systems
Professional: Inquisitive, Extroverted, Effective communication, Adaptability

EXPERIENCE:

Crew Member/Crew Trainer, Panera **06/2020-08/2022**
Brecksville Ohio

- Taking orders
- Handling money
- Serving food
- Preparing food
- Training incoming crew members
- Handling customer complaints

Crew Member/Crew Trainer, CVS **06/2023-09/2023**

- Provided excellent customer service.
- Restocked shelves
- Helping customers of all ages

ACTIVITIES AND INTERESTS:

Brecksville Broadview Heights High School Varsity Track Team **2019-2021**
Brecksville Broadview Heights High School Marching Band **2018-2022**
Indoor Drumline, Brecksville Broadview Heights High School **2018-2022**

Mohammed Arshad

MOHAMMED ARSHAD

776 Euclid Ave, Cleveland, OH 44114

380-240-1086 | m.1nu100@vikes.csuohio.edu | linkedin.com/in/mohammedarshad31 | github.com/Yaya3124

EDUCATION

Cleveland State University,
Bachelors in computer science, Minor in Mathematics

Aug. 2022 – Dec. 2026

TECHNICAL SKILLS

Programming & Frameworks: Java, Python, C/C++, SQL (Postgres), JavaScript, HTML/CSS, SwiftUI, React, Node.js / Express.js, Pandas, NumPy, Matplotlib, Tensorflow / Keras (basic).

Tools and Platforms: Docker, GitHub, VS Code, PyCharm, IntelliJ, Eclipse, XCode, Linux / Bash Scripting, Git CLI, PostgreSQL / MySQL, Jupyter Notebook.

Certification(s): AWS Certified Cloud Practitioner.

EXPERIENCE

Cleveland State University (Office of Research)

May 2025 – Sept 2025

Summer Undergraduate Research Assistant

- Processed large wearable sensor datasets (JSON to Excel), performing data cleaning and feature engineering.
- Conducted exploratory analysis with **Pandas, NumPy, Matplotlib**, identifying links between activity and mood.
- Built and evaluated LSTM models (**TensorFlow / Keras**), improving mood prediction accuracy from ~26% to ~50%, and authored technical documentation and contributed to a research poster presented at CSU symposiums.

Cleveland State University

Aug 2023 – May 2026

Engineering Peer Teacher

- Delivered weekly **Java** and **Python** lectures with interactive coding demos, reaching 30+ students per session.
- Mentored individuals and small groups, boosting average assignment scores, by designing and grading problem sets on recursion, OOP, and data structures with detailed feedback.
- Recognized as **Engineering Peer Teacher of the Month (Feb 2025)** for outstanding engagement.

Cleveland State University

Aug 2024 – Dec 2024

Math Tutor

- Tutored students across algebra, calculus, and discrete math, improving comprehension and exam scores.
- Facilitated group study sessions and tailored strategies for different learning styles.
- Supported exam prep through step-by-step problem walkthroughs and review materials.

PROJECTS

SchedEase — Web-Based Schedule Planner | Java, Vaadin, Full Stack

- Built an interactive planner to visualize and manage weekly class schedules.
- Designed backend data models and integrated Vaadin UI for smooth updates.
- Implemented schedule conflict detection to prevent overlapping classes.
- Delivered final project in a team setting, emphasizing collaboration and reliability.

Mobile UNO Game (in Progress) | React Native, Node.js

- Developing iOS card game with React Native, implementing full UNO ruleset including action/wild cards.
- Built deck generator for 108 cards, managing state transitions and turn based logic.
- Designed multiplayer-friendly game loop with stacking rules and win conditions.
- Using GitHub for version control and iterative feature releases.

RELEVANT COURSEWORK

- | | |
|--------------------------------|-------------------------------|
| • Data Structures & Algorithms | • Object Oriented Programming |
| • Operating Systems | • Computer Networks |
| • Database Systems | • Machine Learning Models |
| • Software Engineering | • Cloud Computing |

Dante Hampton
Maple Heights, OH
eruditehampton14@gmail.com

Education **Cleveland State University / Bachelors of Computer Science**
JAN 2023 - Current, Cleveland

Cuyahoga Community College / Associates of Science
SEPT 2019 - DEC 2022, Cleveland

Experience **Lunabotics**
SEPT 2025 - PRESENT, Cleveland

- NASA competition to create a model lunar rover and compete against other rovers from other universities.
- member of the software sub-team tasked with developing software to operate lunar rover.

Python-Libvirt Virtualization project
Personal Project(2020 - 2022)

- Built a website that used an API to communicate with open-source hypervisor called Libvirt that allowed a user to set up virtual machines on a remote server
- Website was developed using the Python web framework Django
- Used python module pylibvirt in order to communicate with the hypervisor software to allocate physical resources for virtual machines

Eaton - Information Technology intern
SEPT 2018 - MAY 2019

- Cloning data from old SSD's to new SSD's
- Setting up computers, work-stations, and networks for business personnel
- Used python module pylibvirt in order to communicate with the hypervisor software to allocate physical resources for virtual machines

Skills

Software: C, Java, Javascript, Python, MySQL, MongoDB, JSON, Libvirt, Linux, HTML, CSS, Django, [Node.js](#).

Hardware: Arduino(Uno, Uno4), RaspberryPi

SAI NITHIN GUDURU

Ph: +1(216)972-5284

Sainithinguduru@gmail.com

Motivated Computer Science major aiming for a challenging role in software development where I can utilize my programming expertise, strong problem-solving skills, and familiarity with cutting-edge technologies to drive the success of a forward-thinking organization.

Education Qualifications:

- Pursuing Computer Science Engineering, BS from Cleveland State University – **Aug 2021- TILL NOW.**
- Completed High School from FIITJEE College, India – **Jun 2017- June 2019.**

Work Experience:

- Worked at Lead Shuttle Technologies as Junior Software Trainee from **Jan 2020 – July 2021.**
- Worked at Lead Shuttle Technologies as Software Intern from **July 2019- Dec 2019.**

Technical skills:

- Languages: JAVA, HTML, CSS, C, C++
- Databases: SQL, MYSQL.

Project #1:

Academic Project: [Jan 2023- July 2023]

Role: Java

Responsibilities:

Space Invader Game (JavaFX)

- Developed an interactive game using JavaFX where the player controls a rocket to escape from incoming asteroids.
- Implemented object collision detection, game physics, and movement algorithms for smooth gameplay experience.
- Utilized JavaFX for the user interface and graphical rendering of game objects such as the rocket, asteroids, and background.
- Integrated keyboard event handling for player controls and enhanced game interaction.
- Designed multiple levels with increasing difficulty and optimized performance for real-time rendering.

Project #2:

Client Project: GoCanvas [**Jan 2020- July 2021**]

Role: Java Developer Trainee

Description:

GoCanvas is committed to transforming how businesses connect their office and field workers. Their user-friendly platforms aim to simplify inspections, digitize traditional paper-based processes, improve safety, and ensure maximum compliance with industry standards. GoCanvas mobile and cloud solutions are designed to streamline data collection from the field to the office, taking care of administrative tasks.

Responsibilities:

- Implemented agile methodologies to design and execute an intranet project, improving team collaboration efficiency by 35%.
- Leveraged Java SE to build 5+ interactive user interfaces, increasing customer engagement by 50%.
- Applied Test-Driven Development (TDD), reducing bug resolution time by 30%.
- Prepare and produce releases of software components.
- Collaborate with cross-functional teams to define, design, and ship new features, and participate in all phases of the development lifecycle.
- Performed API integration into 2 mobile apps, resulting in increased functionality and user ratings by 15%.

Project #3:

Client Project: IonixxTech [July 2019- Dec 2019]

Role: Software Developer Intern

Description:

As an established healthcare software development company, Ionixx has a rich portfolio of digital healthcare solutions driving patient engagement and automating fastidious tasks in inpatient and outpatient care.

Responsibilities:

- Create detailed design documentation.
- Contributed code to mainline service modules contributing to an overall 20% speed boost.
- Supported debugging operations to resolve 50+ user reported issues, enhancing overall software reliability.
- Participate in Agile development methodologies, including using JIRA for project management.

(SAI NITHIN GUDURU)

Appendix B: Sponsor specification

Psyche Web-Based Games

The following summarizes what we have discussed in our meetings.

- Please message Cassie in Slack if you have questions or need clarifications.
- Please message Suryansh Purohit in Slack if you have technical questions.
- Web-Based Game Portal (previous games):

<https://gameportal.missiontopsyche.org/>

Content of Game:

Please use the Psyche mission website (<https://psyche.ssl.berkeley.edu>), the

Process and Lifetime of a Space Mission course

(<https://psyche.ssl.berkeley.edu/online-course>), and papers about Psyche in the participant folder

(<https://drive.google.com/drive/folders/1naT6BDjApvacQvpSAIgpu2GfXspAEppD>)

for inspiration and information. If other information is needed, feel free to message

Cassie in Slack.

Audience for Game:

The Psyche mission develops materials for all sorts of audiences, but not all of our materials need to target all audiences. You should determine your audience for your game and then design your game to best serve that audience (do some background research--what are the best practices for making games for that audience?).

Purposes/Aims of Game:

You can decide where on the “edutainment” spectrum your game lands. However, it should somehow engage the user in the excitement and innovation of the mission and help them feel connected to the mission. It should NOT introduce or reinforce misconceptions.

Features/Functionality of Game:

The web game should be able to work well on a consumer-grade computer on a standard home Wi-Fi connection. Don’t make something that only people with gaming computers or super-fast Wi-Fi can play/enjoy. It does not have to also be mobile compliant, though it may be if you want the challenge of doing that.

Note: From our perspective, it is better to have a simpler game done really well than something more complicated that you can’t actually complete.

Length of Game:

The length of the game should be determined by best practices for the type of game you are designing and the audience you are designing for.

Deliverables/Milestones:

All members of each team need to meet the deadlines set forth by their capstone course. For interim game deliverables, you should determine an end date for the complete game (sometime before your final semester/term/quarter) and then work backwards in time to determine your milestones. If you want your game to go through review before you graduate, you should provide the game to us by a month before graduation.

Some DON'Ts:

- No shooting/violence.
- Don't introduce or reinforce misconceptions!
 - If you want your game to have something that is not realistic to the current Psyche mission (such as landing on the surface) the game should make it clear that it is taking place in the **far future**. Please discuss this option with Cassie before proceeding. If you are planning to have humans on the surface, please refer to a "community" rather than "colony" or "settlement".
 - A lot of teams want to fly through an asteroid field. Even though that isn't realistic (for example, according to the mission Imager Lead, Psyche won't see another asteroid on its journey!), I've confirmed that it is allowed for the games. You could consider other bodies as well, such as meteoroids.
 - See: <https://solarsystem.nasa.gov/asteroids-comets-and-meteors/meteors-and-meteorites/overview/>

"Meteoroids are objects in space that range in size from dust grains to small asteroids. Think of them as "space rocks."

When meteoroids enter Earth's atmosphere (or that of another planet, like Mars) at high speed and burn up, the fireballs or

“shooting stars” are called **meteors**.

When a meteoroid survives a trip through the atmosphere and hits the ground, it’s called a **meteorite**.”

- No e-commerce
- No accounts or profiles; no way for players to contact each other.
- Don’t infringe on copyrights or misuse licenses.

Acceptance Criteria:

Web-Based Game

- The game should work on all standard browsers (Chrome, Mozilla, Firefox, Edge). However, you can let the user know which browser(s) the game is optimized for.
- The game should NOT crash the user’s browser or cause problems for their computer.
- The game should NOT have security vulnerabilities.
- The game should NOT introduce or reinforce misconceptions.
- The game should be appropriate for its intended audience.

GitHub Code Repository

- Code should be committed with proper comments - it should provide information on the specific feature you committed.
- There should be a README that provides information on how to compile the source code, as well as information on any third-party library or plugins used.
- Please include a build folder (with index.html and compiled project) that can be deployed on a web server.

Code Quality

- Avoid hardcoding, use constants/configuration values, for ex, for the values you want to use as a hard-coded text, you should maintain a separate JSON or text file and read it from that file.

- Write comments on what you are doing and on why you are doing it - also, you can use documentation tools to generate the comments.
- Reusability: Try not to repeat the same code.
- Try to do exception and error handling where applicable.
- Give meaningful names to variables.
- Use a modular coding style - try to separate the functionalities in different modules, for better testing and debugging.
 - Tip: Use GitHub [Issues](#) and [Project Boards](#) to keep track of your work.
- You should try to implement unit testing, so at least 85% of your code is covered.

Graphic Design:

Psyche Brand Guidelines: it is preferred (but not required) to follow the Psyche brand guidelines in terms of main colors, found near the bottom of:

<https://psyche.ssl.berkeley.edu/galleries/> The Psyche badge may be used as featured art or elsewhere in the game.

NASA Image Use Policy: Teams must follow NASA's Image Use policy:

<https://www.nasa.gov/multimedia/guidelines/index.html>

NASA Style Guide: As needed:

https://drive.google.com/file/d/1thyZBC8lYvSEUH9oygpAJO6Rq_zEV0o0/view?usp=drive_link

If help is needed to find or get access to graphics/visual assets, please message Cassie Bowman in Slack.

All graphic design and other visual assets developed by the team for the game must be provided in full at the end of the project.

Written Text:

Approved written text about the Psyche mission is available on the Psyche website <https://psyche.ssl.berkeley.edu>, including the mission FAQs: <https://psyche.ssl.berkeley.edu/mission/faq/>. All text on the website has been approved for use.

Governmental guidelines for public audience materials may be of help: [plainlanguage.gov](https://www.plainlanguage.gov/); [usability.gov](https://www.usability.gov/).

Accessibility:

It is ideal for the game to be as accessible/inclusive as possible. Good tips are available here: <https://developers.google.com/web/fundamentals/accessibility>

For ASU students, there is free training (which results in a digital credential) on Accessibility for Web Developers: <https://canvas.asu.edu/courses/383> (requires ASURITE). Other universities likely offer similar courses.

Documentation and Assets:

The game and its source code must be fully documented. All assets must be provided in full at the end of the project, including the GitHub repository.

Test Server:

Guide to hosting web-based projects: [Website Hosting Solutions](#)

Please reach out to Suryansh Purohit in our Slack if you need help accessing a test server.